

In[0.1]: Instalação dos pacotes

```
!pip install pandas
!pip install numpy
!pip install -U seaborn
!pip install matplotlib
!pip install plotly
!pip install scipy
!pip install statsmodels
!pip install scikit-learn
!pip install statstests
!pip install python-docx
```

In[0.2]: Importação dos pacotes

```
import pandas as pd # manipulação de dados em formato de dataframe
import numpy as np # operações matemáticas
import seaborn as sns # visualização gráfica
import matplotlib.pyplot as plt # visualização gráfica
from math import exp, factorial # funções matemáticas 'exp' e 'factorial'
import statsmodels.api as sm # estimação de modelos
import statsmodels.formula.api as smf # estimação de modelos (logístico binário e contagem)
from statsmodels.discrete.count_model import ZeroInflatedNegativeBinomialP, ZeroInflatedPoisson #
modelos ZINB e ZIP
from statsmodels.discrete.discrete_model import NegativeBinomial, Poisson, MNLogit # modelos NB,
Poisson e logístico multinomial
from scipy import stats # estatística chi2
from scipy.interpolate import UnivariateSpline # curva sigmoide suavizada
from statsmodels.iolib.summary2 import summary_col # comparação entre modelos
from statstests.process import stepwise # procedimento Stepwise
import plotly.graph_objects as go # gráficos 3D
import docx
import warnings
from docx import Document
from docx.shared import Pt
from docx.enum.text import WD_PARAGRAPH_ALIGNMENT
from docx.oxml import OxmlElement
from docx.oxml.ns import qn
warnings.filterwarnings('ignore')
```

CARREGAMENTO

```
dados = pd.read_excel('703pacientes.xlsx')
#dados = pd.read_excel('Covid_crit.xlsx')
dados.info()
#### Tabela de frequências
```

```
colunas = [
    #'Vacinas',
    'Vacinado',
    'Sexo',
    'Estado_Civil_1',
    'Estado_Civil_2',
    'Idade_cat_1',
    'Idade_cat_2',
    'Idade_cat_3',
    'Óbito',
    #'Cod',
    #'Data_entrada',
    'Trombose',
    'Seps',

```

```

'SRAG',
'Choque'
'Grau_Instrucao_1',
'Grau_Instrucao_2',
#'Atend_Profissao',
#'Profissao_categoricas',
'UF',
'Município',
'Fabricante',
'Nenhuma_dose',
'Aztrazeneca',
'Pfizer',
'Coronavac_Butantan',
'Janssen',
'Prob_Card',
'CP',
'Diabetes',
'SRAG',
'Choques',
'Prob_neurol',
'Prob_Hemat',
'Cancer',
'Prob_Resp',
'Prob_Infec',
'Prob_Metab',
'Prob_TGI',
'Prob_Hep',
'Prob_Hid_Elet',
'Prob_AI_Infla',
'Febre',
#'Outros',
'Traumatismo',
'Covid_critica',
'Prob_Renal',
'LRA'
]

```

```

n_total = 720

```

Criar documento Word

```

doc = Document()

```

```

for coluna in colunas:

```

```

    □ req._absoluta = dados[coluna].value_counts().rename('Frequência Absoluta')

```

```

    □ req._relativa = (dados[coluna].value_counts() / n_total * 100).round(2).rename('Frequência Relativa (%)')

```

```

tabela = pd.concat([□ req._absoluta, □ req._relativa], axis=1).reset_index()

```

```

tabela.columns = [coluna, 'Frequência Absoluta', 'Frequência Relativa (%)']

```

Título da tabela (formato ABNT)

```

titulo = doc.add_paragraph()

```

```

run = titulo.add_run(f'Tabela - Frequência da variável {coluna}')

```

```

run.bold = True

```

```

titulo.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER

```

```

titulo.space_after = Pt(12)

```

Criar a tabela no Word

```
table = doc.add_table(rows=1, cols=len(tabela.columns))
table.style = 'Table Grid'
```

Cabeçalho da tabela

```
for i, heading in enumerate(tabela.columns):
    cell = table.cell(0, i)
    cell.text = heading
    # Negrito no cabeçalho
    for paragraph in cell.paragraphs:
        for run in paragraph.runs:
            run.bold = True
```

```
# Adicionar as linhas da tabela
for idx, row in tabela.iterrows():
    row_cells = table.add_row().cells
    for i, item in enumerate(row):
        row_cells[i].text = str(item)
```

Espaçamento após a tabela

```
doc.add_paragraph()
```

Salvar o arquivo Word

```
nome_arquivo = 'frequencias_vaccine.docx'
doc.save(nome_arquivo)
```

Tabela de frequências

```
colunas = [
    'Óbito',
    'Trombose',
    'Seps',
    'SRAG',
    'Choque'
    '#Prob_Card',
    '#CP',
    '#Diabetes',
    '#SRAG',
    '#Choques',
    '#Prob_neurol',
    '#Prob_Hemat',
    '#Cancer',
    '#Prob_Resp',
    '#Prob_Infec',
    '#Prob_Metab',
    '#Prob_TGI',
    '#Prob_Hep',
    '#Prob_Hid_Elet',
    '#Prob_AI_Infla',
    '#Febre',
    '#Outros',
    '#Traumatismo',
    '#Covid_critica',
    '#Prob_Renal',
    '#LRA'
]
```

```
n_total = 720
```

Criar documento Word

```
doc = Document()

for coluna in colunas:
    freq_absoluta = dados[coluna].value_counts().rename('Frequência Absoluta')
    freq_relativa = (dados[coluna].value_counts() / n_total * 100).round(2).rename('Frequência Relativa (%)')

    tabela = pd.concat([freq_absoluta, freq_relativa], axis=1).reset_index()
    tabela.columns = [coluna, 'Frequência Absoluta', 'Frequência Relativa (%)']

    # Título da tabela (formato ABNT)
    titulo = doc.add_paragraph()
    run = titulo.add_run(f'Tabela - Frequência da variável {coluna}')
    run.bold = True
    titulo.alignment = WD_PARAGRAPH_ALIGNMENT.CENTER
    titulo.space_after = Pt(12)
```

Criar a tabela no Word

```
table = doc.add_table(rows=1, cols=len(tabela.columns))
table.style = 'Table Grid'
```

Cabeçalho da tabela

```
for i, heading in enumerate(tabela.columns):
    cell = table.cell(0, i)
    cell.text = heading
    # Negrito no cabeçalho
    for paragraph in cell.paragraphs:
        for run in paragraph.runs:
            run.bold = True
```

Adicionar as linhas da tabela

```
for idx, row in tabela.iterrows():
    row_cells = table.add_row().cells
    for i, item in enumerate(row):
        row_cells[i].text = str(item)
```

Espaçamento após a tabela

```
doc.add_paragraph()
```

Salvar o arquivo Word

```
nome_arquivo = 'frequencias_pacientes_covid_critic.docx'
doc.save(nome_arquivo)
```

RI simples várias

```
import pandas as pd
import statsmodels.formula.api as smf
import statsmodels.api as sm
from docx import Document
```

```
# =====
# 1. LISTA DE VARIÁVEIS INDEPENDENTES
# =====
```

```

variaveis = [
    'Óbito',
    #'Trombose',
    #'Seps',
    'SRAG',
    #'Choque',
    'Prob_Card',
    'CP',
    'Diabetes',
    'SRAG',
    'Choques',
    'Prob_neurol',
    'Prob_Hemat',
    'Cancer',
    'Prob_Resp',
    'Prob_Infec',
    'Prob_Metab',
    'Prob_TGI',
    'Prob_Hep',
    'Prob_Hid_Elet',
    'Prob_AI_Infla',
    'Febre',
    #'Outros',
    'Traumatismo',
    #'Covid_critica',
    'Prob_Renal',
    'LRA',
    #'Trombose', "Seps", "SRAG", "Choque"
    'Vacinado',
    #'Sexo', 'Prob_Card', 'CP', 'Diabetes', 'UF', 'Município', 'SRAG', 'Choques',
    #'Prob_neurol', 'Prob_Hemat', 'Cancer', 'Prob_Resp', 'Prob_Infec', 'Pfizer',
    #'Prob_Metab', 'Prob_TGI', 'Prob_Hep', 'Prob_Hid_Elet', 'Prob_AI_Infla', 'Janssen',
    #'Febre', 'Outros', 'Traumatismo', 'COVID_CRÍTICA', 'Prob_Renal', 'LRA', 'Coronavac_Butantan',
    #'Dias_permanência',
    'Estado_Civil_1',
    'Estado_Civil_2',
    'Idade_cat_1',
    'Idade_cat_2',
    'Idade_cat_3',
    'Fabricante',
    'Nenhuma_dose',
    'Aztrazeneca',
    'Grau_Instrucao_1',
    'Grau_Instrucao_2'
]

```

Variável dependente

y = 'Óbito'

=====

2. LISTA PARA RECEBER OS RESULTADOS

=====

resultados = []

=====

3. RODAR UMA REGRESSÃO PARA CADA VARIÁVEL

=====

for var in variaveis:

formula = f"{y} ~ {var}"

```

modelo = smf.glm(formula=formula, data=dados, family=sm.families.Binomial()).fit()

beta = modelo.params[var]
p = modelo.pvalues[var]
ic_low, ic_high = modelo.conf_int().loc[var]

OR = np.exp(beta)
OR_low = np.exp(ic_low)
OR_high = np.exp(ic_high)

resultados.append({
    'Variável': var,
    'Beta': beta,
    'IC95_low': ic_low,
    'IC95_high': ic_high,
    'p-valor': p,
    'OR': OR,
    'OR_low': OR_low,
    'OR_high': OR_high
})

# =====
# 4. GERAR DATAFRAME FINAL
# =====

pd.set_option('display.max_rows', None) # mostra todas as linhas
pd.set_option('display.max_columns', None) # mostra todas as colunas
pd.set_option('display.width', None) # evita quebra de linha
pd.set_option('display.max_colwidth', None) # evita truncar conteúdo
pd.set_option('display.float_format', '{:.6f}'.format)
df_resultados = pd.DataFrame(resultados)
print(df_resultados)
# =====

### tabela word com os resultados da RL
from docx import Document

# Criar documento Word
document = Document()
document.add_heading("Resultados das Regressões Logísticas", level=1)

# Criar tabela com número correto de colunas
tabela = document.add_table(rows=1, cols=len(df_resultados.columns))

# Preencher cabeçalho
hdr_cells = tabela.rows[0].cells
for i, col in enumerate(df_resultados.columns):
    hdr_cells[i].text = str(col)

# Preencher linhas do dataframe
for idx, row in df_resultados.iterrows():
    row_cells = tabela.add_row().cells
    for i, col in enumerate(df_resultados.columns):
        valor = row[col]

        # arredondamento para números
        if isinstance(valor, float):
            valor = round(valor, 4)

```

```

row_cells[i].text = str(valor)

# Salvar arquivo
document.save("resultados_regressoes_binarias2.docx")

print("Arquivo Word gerado com sucesso!")

#### RL com todas as var independ
modelo_rl = smf.glm(formula='Óbito ~ Vacinado + Pfizer + Sexo + Nenhuma_dose + Prob_Card +
Aztrazeneca + CP + UF + Município + Diabetes + SRAG + Pfizer + Choques + Prob_neurol + Janssen +
Prob_Hemat + Coronavac_Butantan + Cancer + Prob_Resp + Prob_Infec + Prob_Metab + Prob_TGI +
Prob_Hep + Prob_Hid_Elet + Prob_AI_Infla + Febre + Outros + Traumatismo + COVID_CRÍTICA +
Prob_Renal + LRA + Dias_permanência + Estado_Civil_1 + Estado_Civil_2 + Idade_cat_1 + Idade_cat_2
+ Idade_cat_3 + Grau_Instrucao_1 + Grau_Instrucao_2', data=dados, family=sm.families.Binomial()).fit()
#modelo_rl = smf.glm(formula='Óbito ~ Trombose + Sepsis + SRAG + Choque', data=dados,
family=sm.families.Binomial()).fit()

modelo_rl.summary()

##### PARTE COMPLEMENTAR #####
# OR
or_values = np.exp(modelo_rl.params)

# IC 95% dos coeficientes -> transformar em OR também
conf = modelo_rl.conf_int()
conf['OR_inf'] = np.exp(conf[0])
conf['OR_sup'] = np.exp(conf[1])

# Tabela final organizada
resultado = pd.DataFrame({
    'Coef (β)': modelo_rl.params,
    'OR': or_values,
    'IC_inf': conf['OR_inf'],
    'IC_sup': conf['OR_sup'],
    'p-valor': modelo_rl.pvalues
})
pd.set_option('display.float_format', '{:.6f}'.format)
print(resultado)
#%%%
from statstests.process import stepwise

# Estimação do modelo por meio do procedimento Stepwise
step_modelo = stepwise(modelo_rl, pvalue_limit=0.05)

#%%% trocando vacinado por vacinas (pois não posso colocar as duas no mesmo
# modelo devido à multicolinearidade).
dados = pd.get_dummies(dados, columns=['Vacinas'],
                        dtype=int,
                        drop_first=True)

dados.info()
#%%% RL com todas as var independ
modelo_rl_vacinado = smf.glm(formula='Óbito ~ Vacinas_1 + Vacinas_2 + Vacinas_3 + Vacinas_4 + Sexo
+ Prob_Card + CP + Diabetes + SRAG + Choques + Prob_neurol + Prob_Hemat + Cancer + Prob_Resp +
Prob_Infec + Prob_Metab + Prob_TGI + Prob_Hep + Prob_Hid_Elet + Prob_AI_Infla + Febre + Outros +
Traumatismo + COVID_CRÍTICA + Prob_Renal + LRA + Dias_permanência + Estado_Civil_1 +
Estado_Civil_2 + Idade_cat_1 + Idade_cat_2 + Idade_cat_3 + Grau_Instrucao_1 + Grau_Instrucao_2',
data=dados, family=sm.families.Binomial()).fit()

modelo_rl_vacinado.summary()

```

```

# Coeficientes
betas = modelo_rl_vacinado.params

# Odds Ratio
or_values = np.exp(betas)

# Intervalo de confiança ( $\beta$ )
conf = modelo_rl_vacinado.conf_int()

# IC transformado para OR
or_low = np.exp(conf[0])
or_high = np.exp(conf[1])

# Montando tabela final
resultado_vacinas = pd.DataFrame({
    'Variável': betas.index,
    'Beta': betas.values,
    'OR': or_values.values,
    'OR_low': or_low.values,
    'OR_high': or_high.values,
    'p-valor': modelo_rl_vacinado.pvalues.values
})

# Formatação opcional
pd.set_option('display.float_format', '{:.6f}'.format)

print(resultado_vacinas.to_string(index=False))

#%%
from statstests.process import stepwise

# Estimação do modelo por meio do procedimento Stepwise
step_modelo = stepwise(modelo_rl_vacinado, pvalue_limit=0.05)

```